



Lab 5. Enforcing Workflows, Pipeline as Code

By the end of this lab exercise, you should be able to:

- Define branch protection rules on GitHub
- Enforce code reviews via pull requests
- Write a pipeline as code
- Learn the syntax to write declarative Jenkins pipelines
- Create Jenkinsfiles for Java applications
- Launch multi-branch pipelines upon commits to a remote repository

Enforcing Workflow with Branch Policies

Resources:

- [Web: Guide to Trunk Based Development](#)
- [Video: Trunk Based Development Explained](#)

Let's create branch policies based on the trunk-based development workflow. With trunk-based development:

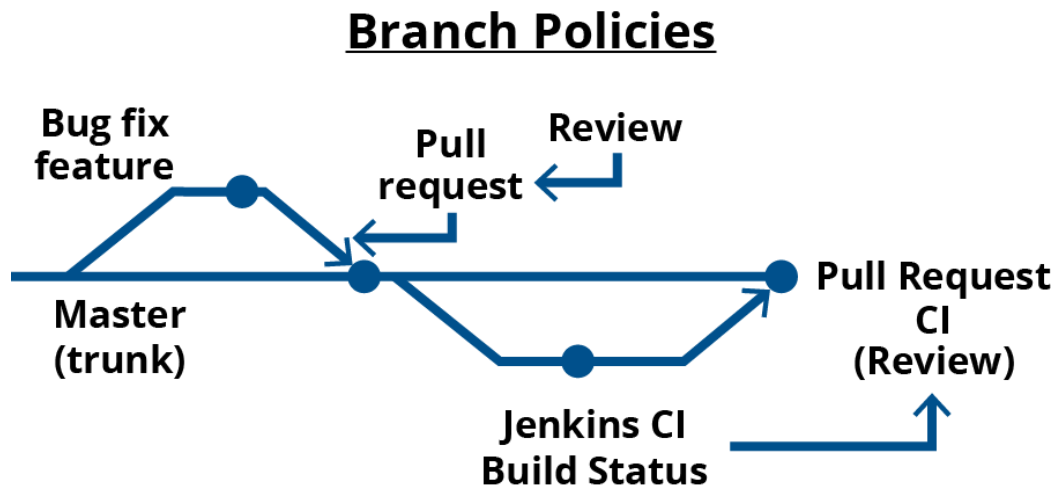
- The master branch is the trunk/main line of code, which is locked (no direct check-ins to master are allowed).
- Every bug fix gets its own branch.
- Every feature gets its own branch too.
- Feature branches are typically short-lived.
- Merges to master require pull requests.
- Further policies could be added to enforce code review, as well as integrate with Jenkins CI to run per branch pipeline.

Make sure you have cloned your own fork of the **example-voting-app** repository locally

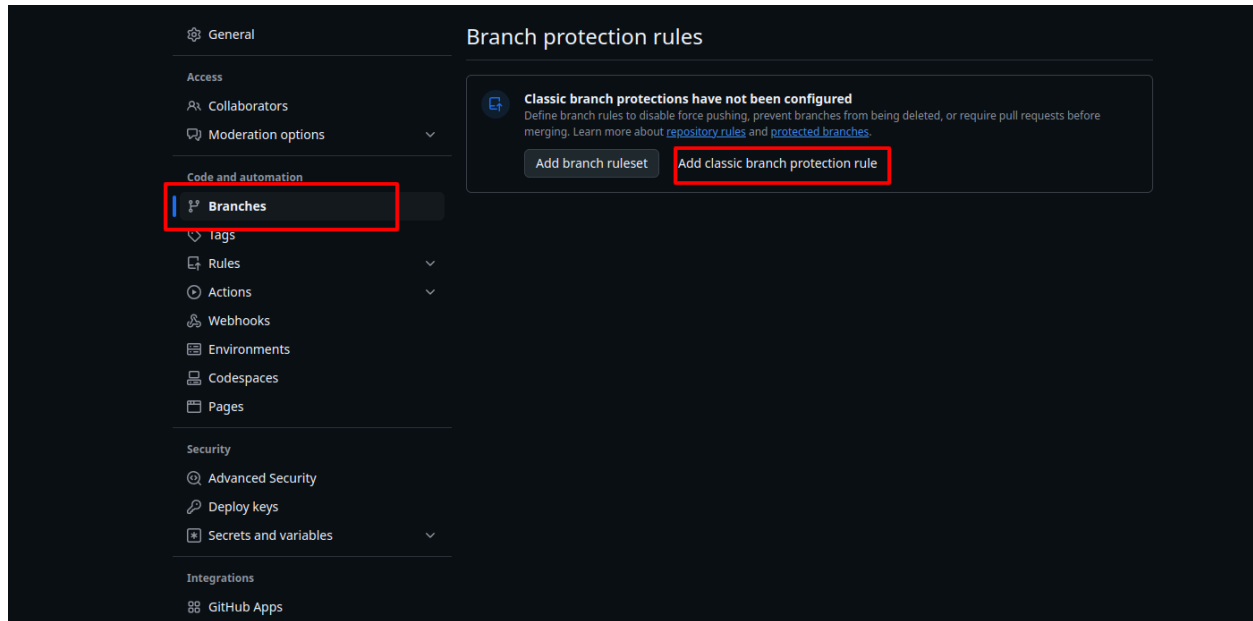
before you proceed. Make sure you are currently inside the `example-voting-app` directory.

NOTE: If you do not have a collaborator, create another account on GitHub, add that account as a collaborator on the `example-voting-app` repository, and add it as a reviewer or skip to the [Writing Pipeline as Code](#) section.

This is how the process looks:



To define the branch policies, go to **Settings** for your `example-voting-app` repository on GitHub and select the **Branches** option.



From the **Branch Protection Rules** section, click on the **Add branch protection rule** button. Add the following set of rules:

- Require pull request reviews before merging
- Dismiss stale pull request approvals when new commits are pushed
- Require status checks to pass before merging
- Do not allow bypassing the above settings

The following screenshots depict the branch protection rules:

Make the name the same as the branch, in this case **master**, that you want to add rules to.

Branch name pattern ^{*}

master

Applies to 1 branch

master

Protect matching branches

☒ **Require a pull request before merging**
When enabled, all commits must be made to a non-protected branch and submitted via a pull request before they can be merged into a branch that matches this rule.

☒ **Require approvals**
When enabled, pull requests targeting a matching branch require a number of approvals and no changes requested before they can be merged.

Required number of approvals before merging: 1 ▼

☒ **Dismiss stale pull request approvals when new commits are pushed**
New reviewable commits pushed to a matching branch will dismiss pull request review approvals.

☐ **Require review from Code Owners**
Require an approved review in pull requests including files with a designated code owner.

☐ **Restrict who can dismiss pull request reviews**
Specify people, teams, or apps allowed to dismiss pull request reviews.

☐ **Allow specified actors to bypass required pull requests**
Specify people, teams, or apps who are allowed to bypass required pull requests.

☐ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.

☐ **Require status checks to pass before merging**
Choose which [status checks](#) must pass before branches can be merged into a branch that matches this rule. When enabled, commits must first be pushed to another branch, then merged or pushed directly to a branch that matches this rule after status checks have passed.

☐ **Require conversation resolution before merging**
When enabled, all conversations on code must be resolved before a pull request can be merged into a branch that matches this rule.
[Learn more.](#)

☐ **Require signed commits**
Commits pushed to matching branches must have verified signatures.

☐ **Require linear history**
Prevent merge commits from being pushed to matching branches.

☐ **Require deployments to succeed before merging**
Choose which environments must be successfully deployed to before branches can be merged into a branch that matches this rule.

☐ **Lock branch**
Branch is read-only. Users cannot push to the branch.

☒ **Do not allow bypassing the above settings**
The above settings will apply to administrators and custom roles with the "bypass branch protections" permission.

☐ **Restrict who can push to matching branches**
Specify people, teams, or apps allowed to push to matching branches. Required status checks will still prevent these people, teams, and apps from merging if the checks fail.

Rules applied to everyone including administrators

☐ **Allow force pushes**
Permit force pushes for all users with push access.

☐ **Allow deletions**
Allow users with push access to delete matching branches.

Save changes

Testing Your New Branch Protection Rules

To test the branch protection rules, from the local repository, make some changes to any file, e.g. `README.md`. Try adding it to Git and commit those changes:

```
git diff
```

```
git commit -am "test"
```

```
git push origin master
```

When you try to push the changes to a remote, it throws an error *"master protected branch hook declined"*. That's because you can't directly push to master anymore.

You can undo the changes by resetting the commit:

```
git log
```

```
git reset --hard HEAD~1
```

Once you do a hard reset, it will roll back the changes you have made to the commit history and also to the local workspace.

Code Reviews with Pull Requests

Next, you will learn how to make changes in the master branch by using code reviews and pull requests. Update the existing `README.md` file for the project and incorporate those changes to the trunk i.e. master branch.

Create a branch `readme` using the following command:

```
git checkout -b readme
```

Running this command will create the `readme` branch, and then switch to it. Update the `README.md` file by adding a description of the application.

For example,

```
filename: README.md
```

```
Example Voting App
```

```
=====
```

```
This is a sample voting app.
```

Save and commit the changes you have made:

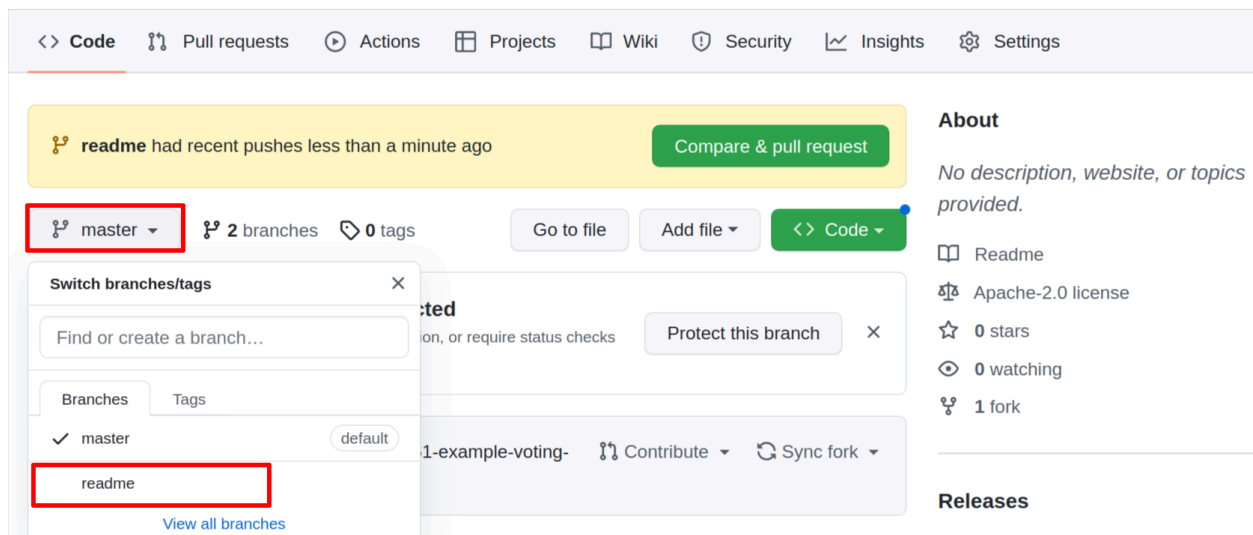
```
git status
```

```
git commit -am "added application info"
```

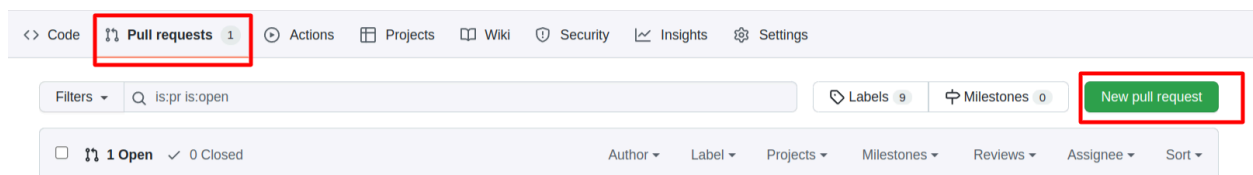
```
git push origin readme
```

This pushes the changes to the `readme` branch of the repo on GitHub.

Go to the GitHub repository and switch to the branch `readme`. You can do this by clicking the dropdown menu that has **master** currently selected:



Select the **Pull requests** tab next to the **Code** tab across the top of the repository:



You should see **New pull request** to the right. The pull request is to allow you to merge the changes from the feature branch on GitHub to the master branch on GitHub.

Select **New pull request** and choose your repository for the **base repository** and choose **master** for the **base**. Select your repository from the **head repository** dropdown and select the `readme` branch in the **Compare** dropdown. Click **Create pull request**.

Comparing changes

Choose two branches to see what's changed or to start a new pull request. If you need to, you can also [compare across forks](#).

 base: master ← compare: readme ✓ **Able to merge.** These branches can be automatically merged.

Discuss and review the changes in this comparison with others. [Learn about pull requests](#) Create pull request

Enter the title and leave a comment if you'd like, then click **Create pull request**:

 Pull

Write Preview

H B I         

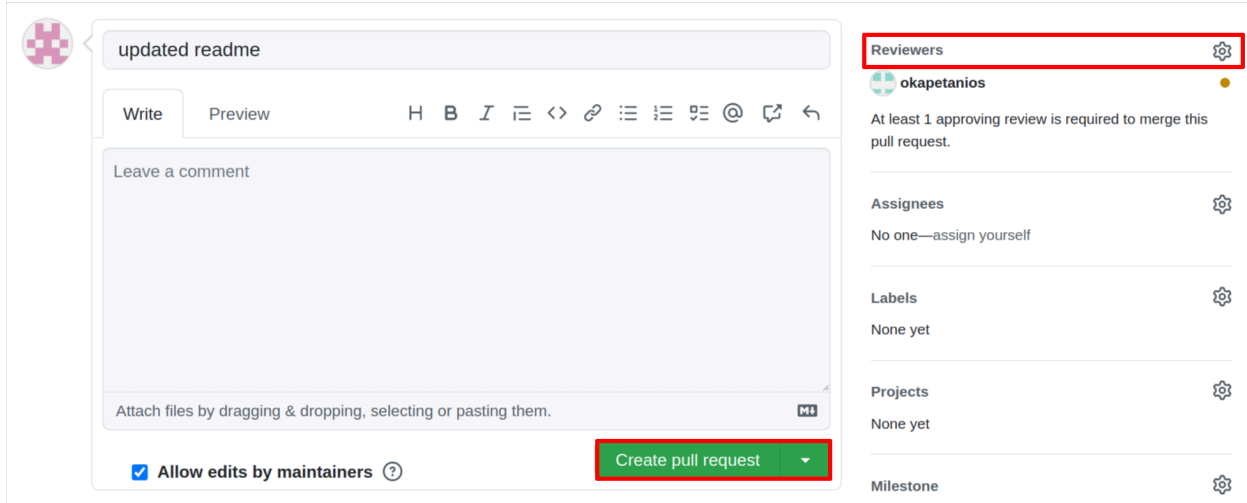
Leave a comment

Attach files by dragging & dropping, selecting or pasting them.

☒ Allow edits by maintainers 

Create pull request

Add a reviewer: Creating a pull request does not allow you to merge immediately. It will require a code reviewer. This is mandated as part of the branch protection rules you created. Add at least one of your collaborators as a reviewer with the **Reviewers** gear icon:



updated readme

Write Preview H B I

Leave a comment

Attach files by dragging & dropping, selecting or pasting them.

☒ Allow edits by maintainers

Create pull request

Reviewers

okapetanio

At least 1 approving review is required to merge this pull request.

Assignees

No one—assign yourself

Labels

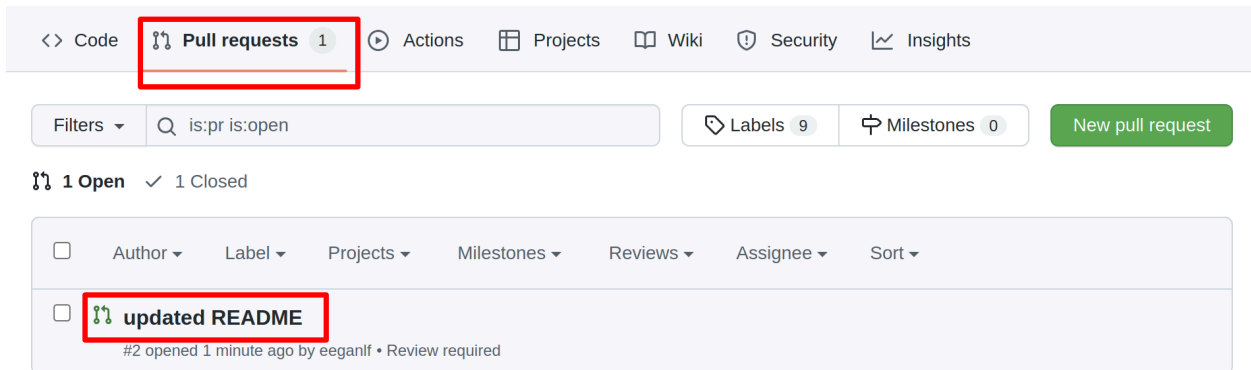
None yet

Projects

None yet

Milestone

Approve a pull request as the reviewer. With the reviewer's account, visit the repository page, go to pull requests, and click on the pull request:



<> Code **Pull requests** 1 Actions Projects Wiki Security Insights

Filters is:pr is:open Labels 9 Milestones 0 New pull request

1 Open 1 Closed

☐	Author	Label	Projects	Milestones	Reviews	Assignee	Sort
☐	updated README						

#2 opened 1 minute ago by eeganlf • Review required

Review the changes, leave a comment if you wish, and approve the request by checking **Approve** and clicking **Submit Review**:

updated README #2

Open eeganlf wants to merge 1 commit into master from readme

Conversation 0 Commits 1 Checks 0 Files changed 2 +45 -0

Changes from all commits File filter Conversations 0 / 2 files viewed Review changes

Filter changed files

README.md

worker

Jenkinsfile

Finish your review

Write Preview

Leave a comment

Attach files by dragging & dropping, selecting or pasting them.

☐ Comment
Submit general feedback without explicit approval.

☒ Approve
Submit feedback and approve merging these changes.

☐ Request changes
Submit feedback that must be addressed before merging.

Submit review

```
4 + image "maven:3.6.1-jdk-8-slim"
5 + args '-v $HOME/.m2:/root/.m2'
```

After approval, merge the pull request to master either from the reviewer's account or the requester's account. Go to **Pull requests > Conversation** and click **Merge pull request** as follows:

The screenshot shows a GitHub Pull Request for the repository 'eeganlf/LFS261-example-voting-app'. The pull request is titled 'updated README #2' and is from the 'readme' branch to the 'master' branch. The 'Conversation' tab is selected, showing a comment from 'eeganlf' and an approval from 'okapetanios'. The 'Changes approved' section is highlighted, indicating that the pull request is ready to be merged. The 'Merge pull request' button is also visible.

Delete the `readme` branch after the merge.

Writing Pipeline as Code

Resources:

- [Declarative Pipeline Syntax](#)
- [Declarative Pipeline Steps](#)

Some of the important instructions are:

- Pipeline
- Agent
- Tools
- Stages, Stage, Steps
- Post

Jenkinsfile for a Java Application

In this section, you will define pipeline as code for a Java worker app which uses Maven as a build tool.

Create a new `feature/workerpipe` branch by using the following commands:

```
git checkout -b feature/workerpipe
```

```
git branch
```

Now you are going to write a Jenkinsfile for the `worker` application.

Create a Jenkinsfile inside the `worker` directory of your code. Before you start writing the code, check your Maven version in your Jenkins: **Manage Jenkins > System Configuration > Tools > Maven Installations** and note down the exact name in the configuration (e.g. Maven 3.9.8).

```
pipeline {
  agent any

  tools{
    maven 'maven 3.9.8'
  }

  stages{
    stage("build"){
      steps{
        echo 'Compiling worker app'
        dir('worker'){
          sh 'mvn compile'
        }
      }
    }
    stage("test"){
      steps{
        echo 'Running Unit Tests on worker app'
      }
    }
  }
}
```

```
    }
    stage("package") {
        steps{
            echo 'Packaging worker app'
        }
    }
}

post{
    always{
        echo 'Building multibranch pipeline for worker is completed..'
    }
}
}
```

Once you create the Jenkinsfile, commit the file and push the changes to **feature/workerpipe**:

```
git status
```

```
git add worker/Jenkinsfile
```

```
git commit -am "added Jenkinsfile for worker with build job"
```

```
git push origin feature/workerpipe
```

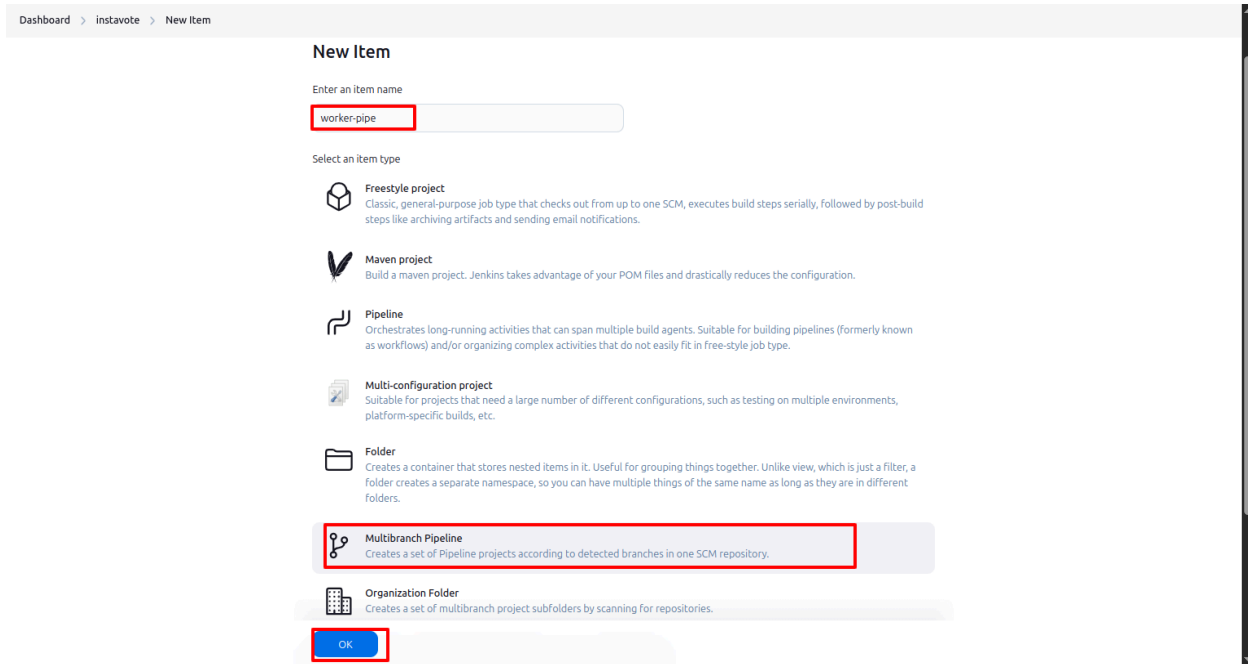
Multi-Branch Pipeline Project

In this section, you are going to create and execute a multi-branch pipeline which:

- Scans your repository automatically for a Jenkinsfile
- Sets up the pipeline jobs for the branches which contain the above pipeline script
- Starts executing the CI jobs

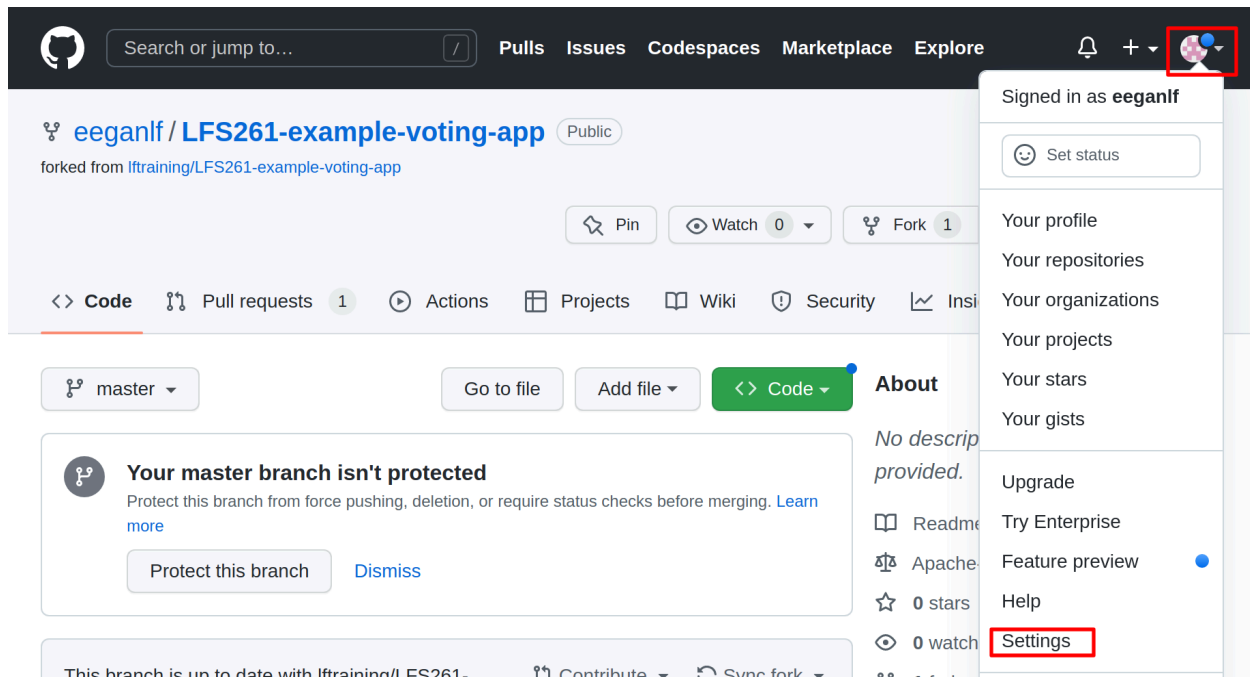
Refer to the following steps to create a multi-branch pipeline.

Create a new job `worker-pipe` in your `instavote` directory; select “Multibranch Pipeline” as the project type.



In the job configuration page, under the branch sources choose “GitHub”, add your account credentials, and choose the project repository. Refer to the images below for exact steps.

In a separate browser, navigate to **GitHub > Settings** and select **Developer Settings**.



Security

Code security and analysis

Integrations

Applications

Scheduled reminders

Archives

Security log

Sponsorship log

<> Developer settings

Generate a *personal access token* with access to **repo** enabled.

The screenshot shows the GitHub 'Personal access tokens (classic)' page. On the left sidebar, 'Personal access tokens' is selected, and 'Tokens (classic)' is highlighted. A modal titled 'Generate new token (classic)' is open, showing options for 'Fine-grained, repo-scoped' and 'For general use'. The 'Generate new token (classic)' option is selected. Below the modal, the 'New personal access token (classic)' form is visible. It includes a 'Note' field with the value 'jenkins-github', an 'Expiration' dropdown set to '90 days', and a 'Select scopes' section with the 'repo' scope checked. The 'repo' scope is described as 'Full control of private repositories'.

Personal access tokens (classic) Generate new token ▼ Revoke all

Tokens you have generated

Generate new token (Beta)
Fine-grained, repo-scoped

Generate new token (classic)
For general use

New personal access token (classic)

Personal access tokens (classic) function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#).

Note
jenkins-github
What's this token for?

Expiration *
90 days ▼ The token will expire on Sat, Feb 11 2023

Select scopes
Scopes define the access for personal tokens. [Read more about OAuth scopes](#).

☒ **repo** Full control of private repositories

☒ repo:status Access commit status

☒ repo_deployment Access deployment status

☒ public_repo Access public repositories

☒ repo:invite Access repository invitations

☒ security_events Read and write security events

Click **Generate token** at the bottom of the page.

Copy the token:

The screenshot shows the GitHub 'Personal access tokens (classic)' page after generating a token. A blue notification box states: 'Make sure to copy your personal access token now. You won't be able to see it again!'. Below this, the generated token is displayed: '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'. The token is highlighted with a red box. A 'Delete' button is visible next to the token.

Personal access tokens (classic) Generate new token ▼ Revoke all

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

✓ 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 📋 Delete

Return to the **worker-pipe** configuration page in Jenkins and configure the credentials. Be sure to use “Username and Password” and provide the token as the password, not the password you use to login to GitHub.

The screenshot shows the Jenkins Configuration page for the **worker-pipe** job. The breadcrumb trail is **Dashboard > instavote > worker-pipe >**. On the left, the **Configuration** sidebar has **Branch Sources** highlighted with a red box. The main content area is titled **Branch Sources** and features an **Add source** button (also highlighted with a red box) that has opened a dropdown menu. The menu options are **Bitbucket**, **Git**, **GitHub** (highlighted with a red box), and **Single repository & branch**. Below the menu, the **Mode** is set to **by Jenkinsfile**. The **Script Path** is **worker/Jenkinsfile**. At the bottom, there are **Save** and **Apply** buttons.

This screenshot shows the **GitHub Credentials** configuration dialog within the **Branch Sources** section. The **Credentials** dropdown is set to **- none -**. The **+ Add** button (highlighted with a red box) has opened a menu where **Jenkins** (highlighted with a red box) is selected. The **Repository HTTPS URL** field is empty. A **Validate** button is present. At the bottom, there is a **Repository Scan - Deprecated Visualization** option and **Behaviors** section with **Save** and **Apply** buttons.

Domain
Global credentials (unrestricted) ▼

Kind
Username with password ▼

Scope ?
Global (Jenkins, nodes, items, all child items, etc) ▼

Username ?
eeganlf

☐ Treat username as secret ?

Password ?

ID ?
eeganlf-github

Description ?
eeganlf-github

Add Cancel

Under **Branch Sources** click **Add source** and select **GitHub**.

GitHub ✕

Credentials ?
eeganlf/***** (eeganlf github) ▼

+ Add ▼

☒ Repository HTTPS URL

Repository HTTPS URL ?
https://github.com/eeganlf/LFS261-example-voting-app.git

Credentials ok. Connected to https://github.com/eeganlf/LFS261-example-voting-app. **Validate**

☐ Repository Scan - Deprecated Visualization

Under **Build Configuration**, input your Jenkinsfile path as `worker/Jenkinsfile` inside the **Script Path** input box.

The screenshot shows the Jenkins Configuration page with the 'Build Configuration' tab selected. The left sidebar contains a list of configuration categories: General, Branch Sources, Build Configuration (highlighted with a red box), Scan Multibranch Pipeline Triggers, Orphaned Item Strategy, Appearance, and Health metrics. The main content area is titled 'Build Configuration' and includes a 'Mode' dropdown set to 'by Jenkinsfile' and a 'Script Path' input field containing 'worker/Jenkinsfile' (highlighted with a red box). Below this is the 'Scan Multibranch Pipeline Triggers' section.

Select “1 minute” from the **Interval** drop down under **Scan Multibranch Pipeline Triggers** and **Save** the configuration.

The screenshot shows the Jenkins Configuration page with the 'Scan Multibranch Pipeline Triggers' tab selected. The left sidebar contains a list of configuration categories: General, Branch Sources, Build Configuration, and Scan Multibranch Pipeline Triggers (highlighted with a red box). The main content area is titled 'Scan Multibranch Pipeline Triggers' and includes a checkbox labeled 'Periodically if not otherwise run' (checked and highlighted with a red box) and an 'Interval' dropdown menu set to '1 minute' (highlighted with a red box). Below this is the 'Orphaned Item Strategy' section.

Status

Configure

Scan Repository Now

Scan Repository Log

View as plain text

Multibranch Pipeline Events

Delete Multibranch Pipeline

People

Build History

Project Relationship

Check File Fingerprint

Move

Open Blue Ocean

Rename

Config Files

Pipeline Syntax

Credentials

Build Queue

No builds in the queue.

Scan Repository Log

Started

[Sun Nov 13 22:29:20 UTC 2022] Starting branch indexing...

22:29:20 Connecting to <https://api.github.com> using eeganlf/***** (auth token for github)

Examining [eeganlf/LFS261-example-voting-app](#)

Checking branches...

Getting remote branches...

Checking branch [master](#)

Getting remote pull requests...

'worker/Jenkinsfile' not found

Does not meet criteria

Checking branch [readme](#)

2 branches were processed

Checking pull-requests...

Checking pull request [#2](#)

'worker/Jenkinsfile' found

Met criteria

Scheduled build for branch: PR-2

1 pull requests were processed

Finished examining eeganlf/LFS261-example-voting-app

[Sun Nov 13 22:29:22 UTC 2022] Finished branch indexing. Indexing took 1.9 sec

Finished: SUCCESS

Dashboard > instavote > worker-pipe >

worker-pipe

Disable Multibranch Pipeline

Branches (2) Pull Requests (0)

S	W	Name	Last Success	Last Failure	Last Duration	Fav
		master	51 sec #1	N/A	39 sec	
		readme	51 sec #1	N/A	39 sec	

Icon: S M L

Atom feed for all

Atom feed for failures

Atom feed for just latest builds

The screenshot displays the Jenkins web interface. On the left, a sidebar contains navigation links: Status, Changes, Build Now, View Configuration, Full Stage View, Open Blue Ocean, GitHub, and Pipeline Syntax. The main content area is titled 'Branch readme' and shows the full project name as 'instavote/worker-pipe/readme'. Below this, there are two artifacts: 'worker-jar-with-dependencies.jar' (4.83 MB) and 'worker.jar' (4.28 KB), each with a 'view' link. The 'Stage View' section shows a table of stage times for a build labeled '#1' on Nov 13, 2022, at 10:32 PM. The table has five columns: 'Declarative: Checkout SCM', 'build', 'test', 'package', and 'Declarative: Post Actions'. The row for build #1 shows times of 1s, 11s, 7s, 10s, and 226ms respectively. Below the table, there are 'Permalinks' for the build, including links for the last build, last stable build, last successful build, and last completed build, all dated '1 min 34 sec ago'.

Stage	Declarative: Checkout SCM	build	test	package	Declarative: Post Actions
Average stage times:	1s	11s	7s	10s	226ms
#1	1s	11s	7s	10s	226ms

It will automatically scan your repository. Whenever it detects a new branch under the repository, it runs the pipeline build automatically.

Once the pipeline run is successful, add two more stages in the same Jenkinsfile that contain two more commands: `mvn clean test` and `mvn package`:

```
pipeline {
    agent any

    tools{
        maven 'maven 3.9.8'
    }

    stages{
        stage("build") {
            steps{
                echo 'Compiling worker app..'
                dir('worker'){
                    sh 'mvn compile'
                }
            }
        }
        stage("test") {
            steps{
                echo 'Running Unit Tests on worker app..'
                dir('worker'){
```

```

        sh 'mvn clean test'
    }
}
stage("package"){
    steps{
        echo 'Packaging worker app'
        dir('worker'){
            sh 'mvn package'
        }
    }
}
post{
    always{
        echo 'Building multibranch pipeline for worker is completed..'
    }
}
}

```

After making changes in Jenkinsfile, commit the file from your `feature/workerpipe` and push the changes into the same branch:

```
git status
```

```
git add worker/Jenkinsfile
```

```
git commit -am "added Test and package job for worker pipeline"
```

```
git push origin feature/workerpipe
```

Once you push the changes to the `feature/workerpipe` branch, the scan will automatically trigger the `feature/workerpipe` branch build in the `instavote` multibranch pipeline.

After running the package, it will create a jar/war file. You need to archive that artifact.

```

pipeline {
    agent any

    tools{
        maven 'maven 3.9.8'
    }

    stages{
        stage("build"){

```

```
        steps{
            echo 'Compiling worker app..'
            dir('worker'){
                sh 'mvn compile'
            }
        }
    }
    stage("test"){
        steps{
            echo 'Running Unit Tests on worker app..'
            dir('worker'){
                sh 'mvn clean test'
            }
        }
    }

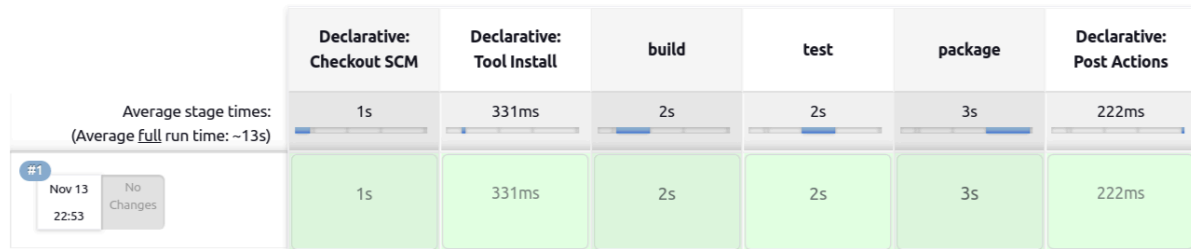
    stage("package"){
        steps{
            echo 'Packaging worker app'
            dir('worker'){
                sh 'mvn package -DskipTests'
            }
        }
    }
}

post{
    always{
        archiveArtifacts artifacts: '**/target/*.jar', fingerprint:
true
    }
    echo 'Building multibranch pipeline for worker is completed..'
}
}
```

Branch feature/workerpipe

Full project name: instavote/worker-pipe/feature%2Fworkerpipe

Stage View



Permalinks

- [Last build \(#1\), 4 min 37 sec ago](#)
- [Last stable build \(#1\), 4 min 37 sec ago](#)
- [Last successful build \(#1\), 4 min 37 sec ago](#)
- [Last completed build \(#1\), 4 min 37 sec ago](#)

Commit the changes to `feature/workerpipe` and push to the branch:

```
git status
```

```
git add worker/Jenkinsfile
```

```
git commit -am "archive artifacts, skip test and package"
```

```
git push origin feature/workerpipe
```

You have set up a multibranch pipeline so that the CI pipelines are run automatically for every branch that you create in future. This is extremely useful to get the feedback from the CI system, before you merge the code into the trunk. This helps ensure that the master branch is never broken.

Configuring Conditional Execution of Stages

With the current configuration, the pipeline you have created will launch for any and every change committed to the repository. However, it makes sense to restrict this behavior to changes made for the `worker` application. This is a desirable feature with mono repositories that contain multiple subprojects.

You may also want to restrict certain jobs to run only on specific branches. For example, integration and acceptance tests should run only for changes to the master branch, and

packaging applications to create and distribute artifacts should only run for the master branch.

This is achieved by setting up conditional execution. You are going to define the following conditions for the **worker** pipeline:

- Run packaging jobs only on master
- Run the jobs in this pipeline only if the code in the **worker** subdirectory is updated

Read the description of the **when** directive in the [Jenkinsfile documentation](#) for reference, then refactor the code as follows:

```
pipeline {
  agent any

  tools{
    maven 'maven 3.9.8'
  }
  stages{
    stage("build") {
      when{
        changeset "**/worker/**"
      }

      steps{
        echo 'Compiling worker app..'
        dir('worker'){
          sh 'mvn compile'
        }
      }
    }
    stage("test") {
      when{
        changeset "**/worker/**"
      }
      steps{
        echo 'Running Unit Test on worker app..'
        dir('worker'){
          sh 'mvn clean test'
        }
      }
    }
    stage("package") {
      when{
```

```

        branch 'master'
        changeset "**/worker/**"
    }
    steps{
        echo 'Packaging worker app'
        dir('worker'){
            sh 'mvn package -DskipTests'
            archiveArtifacts artifacts: '**/target/*.jar',
fingerprint: true }
        }
    }
}

post{
    always{
        echo 'Building multibranch pipeline for worker is completed..'
    }
}
}

```

Once you make changes, commit the changes to `feature/workerpipe` and push to GitHub:

```
git status
```

```
git add worker/Jenkinsfile
```

```
git commit -am "run package step only on master, run stages only when
worker changes"
```

```
git push origin feature/workerpipe
```

After making these changes, you will notice that the build will run only for the first two stages and skip the package stage.

WARNING: Even though `Jenkinsfile` is part of the `worker` subpath, it's not considered to be part of the `changeset` to trigger the builds. You will need to make changes to other files in order to see the jobs being triggered.

You can make some changes in the `README.md` file in the root directory, commit those changes, and push to `feature/workerpipe`. The build will be triggered, but it will skip all three stages.

Use `git log` and `reset` to go back to your previous commit:

```
git log
```

```
git reset --hard yourlastcommitid
```

```
git push origin feature/workerpipe -f
```

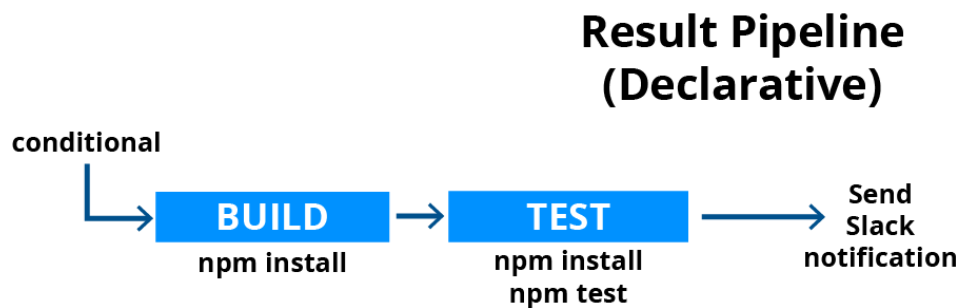
You have learned how to use conditional pipeline stages.

Once you are done with the work, don't forget to incorporate these changes into the master branch via a **pull request** and delete the feature branch.

Exercise: Writing a Jenkinsfile for the Result NodeJS Application

You have been tasked to write a declarative pipeline for the `result` application:

- Two stages, build and test
- Stages should run conditionally only when there is a change to the `result` subdirectory
- You can refer to the following workflow:



The solution to this exercise will be provided as a separate document. Refer to it to validate once you have attempted it yourself.

Summary

We learned how to write a pipeline using declarative code in this lab. We also learned how to enforce workflows through GitHub, create multi-branch pipelines, and add conditional stages.